

<https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types>.

The numeric dunder methods allow objects of your classes to use Python's built-in math operators. If you're writing methods with names like `multiplyBy()`, `convertToInt()`, or something similar that describes a task typically done by an existing operator or built-in function, use the numeric dunder methods (as well as the reflected and in-place dunder methods described in the next two sections).

Reflected Numeric Dunder Methods

Python calls the numeric dunder methods when the object is on the left side of a math operator. But it calls the *reflected* numeric dunder methods (also called the *reverse* or *right-hand* dunder methods) when the object is on the right side of a math operator.

Reflected numeric dunder methods are useful because programmers using your class won't always write the object on the left side of the operator, which could lead to unexpected behavior. For example, let's consider what happens when `purse` contains a `WizCoin` object, and Python evaluates the expression `2 * purse`, where `purse` is on the right side of the operator:

1. Because `2` is an integer, the `int` class's `__mul__()` method is called with `purse` passed for the `other` parameter.
2. The `int` class's `__mul__()` method doesn't know how to handle `WizCoin` objects, so it returns `NotImplemented`.
3. Python doesn't raise a `TypeError` just yet. Because `purse` contains a `WizCoin` object, the `WizCoin` class's `__rmul__()` method is called with `2` passed for the `other` parameter.
4. If `__rmul__()` returns `NotImplemented`, Python raises a `TypeError`.

Otherwise, the returned object from `__rmul__()` is what the `2 * purse` expression evaluates to.

But the expression `purse * 2`, where `purse` is on the left side of the operator, works differently: